

Breaking the Memory Wall: Phase Gradient Flow for Exact Analytical Differentiation

Anonymous Authors¹

Abstract

Training sequence models at extreme contexts ($L > 10^5$) is fundamentally bottlenecked by the memory wall of reverse-mode automatic differentiation (BPTT), which requires storing the entire sequence of intermediate activations and computational graphs. Existing solutions typically trade computation for memory via checkpointing or impose structural constraints. We propose *Phase Gradient Flow (PGF)*, an exact analytical differentiation framework that achieves $\mathcal{O}(1)$ graph memory complexity in both sequence length and network depth. PGF leverages the observation that the derivative dynamics of linear recurrences are analytically isomorphic to the primal system, allowing gradient sensitivity to be propagated purely via the forward state. To instantiate this across deep architectures, we introduce the *Tiled Operator-Space Evolution (TOSE)* protocol. By severing the topological dependency of the automatic differentiation engine and routing adjoints as pure numerical payloads across blocks and layers, TOSE eliminates the AD-tax in both the temporal and depth dimensions. Empirical evaluations demonstrate that PGF yields bit-exact gradients matching BPTT, perfectly overlapping training trajectories, and strictly flat memory scaling, enabling the training of deep sequence models on extreme context lengths without out-of-memory failures.

1. Introduction

Selective State Space Models (SSMs) (Gu & Dao, 2024a;b) enable long-sequence processing, but reverse-mode AD still stores full trajectories, imposing an $\mathcal{O}(L \cdot D \cdot N)$ memory wall. PGF lifts this barrier and enables long-context gradient

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

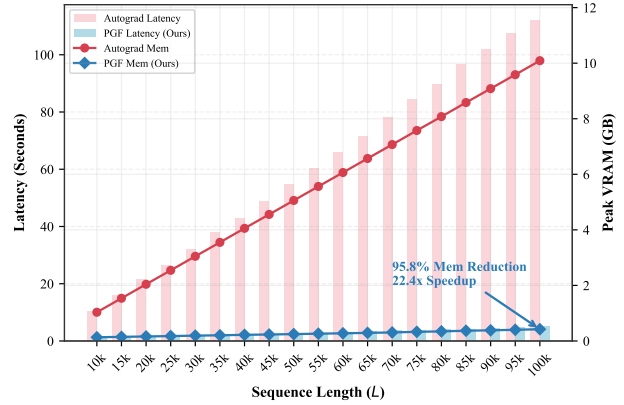


Figure 1. Comparative Video Random Access Memory (VRAM) scaling up to $L = 10^5$. Conventional reverse-mode AD (BPTT; PyTorch Autograd) is bound by linear state retention, breaching 10GB for a single layer. In contrast, PGF enforces an asymptotic memory floor: the differentiation-specific graph memory remains flat ($\mathcal{O}(1)$) and governed by block size B , while total VRAM still includes the unavoidable $\mathcal{O}(L \cdot D)$ I/O payload. This enables long-context gradient computation under fixed VRAM budgets.

computation under fixed VRAM budgets.

This limitation reflects a classical zero-sum trade-off in automatic differentiation (Griewank & Walther, 2008; Baydin et al., 2018). Pure forward-mode AD (JVP) can run with $\mathcal{O}(1)$ memory in L but costs $\mathcal{O}(|\Theta|)$ time to obtain parameter gradients (one directional pass per parameter direction). Pure reverse-mode AD (BPTT; as used by PyTorch Autograd) obtains all parameter gradients in an $\mathcal{O}(1)$ multiple of the forward cost but requires $\mathcal{O}(L)$ memory to retain the unrolled graph. PGF resolves this by acting as a *hybrid AD* scheme: forward-mode structure for recurrent dynamics plus TOSE-based compute-graph detachment to collapse differentiation overhead to $\mathcal{O}(1)$ graph memory.

For linear recurrences, the Fréchet derivative is isospectral with the primal dynamics, so gradients depend only on the *current state*. *Phase Gradient Flow (PGF)* realizes this as a companion forward-mode system, and *Tiled Operator-Space Evolution (TOSE)* enforces $\mathcal{O}(1)$ graph memory by chunking and graph detachment, without activation recom-

putation.

We evaluate PGF on long-sequence sensitivity and training comparisons, showing near machine-precision input gradients and overlapping loss curves under matched initialization and data. Extreme-length survival tests at $L = 100,000$ remain stable under constant peak VRAM, while reverse-mode AD (BPTT) baselines fail due to sequence-length-dependent memory growth.

Contributions are threefold: (i) a history-free differentiation principle for GLRs via PGF; (ii) a graph-memory transition from $O(L)$ to $O(1)$ under TOSE (payload $O(LD)$ unchanged); and (iii) empirical evidence of large VRAM reductions, speedups, and matched training dynamics.

2. Related Work

2.1. General Linear Recurrences and State Space Models

Selective SSMs, evolving from *S4* (Gu et al., 2022) to *Mamba* (Gu & Dao, 2024a) and *Mamba-2 (SSD)* (Gu & Dao, 2024b), enable long-sequence modeling via input-dependent transitions. This family extends to long-convolutions (Hyena (Poli et al., 2023)) and linearized attention (RetNet (Sun et al., 2023), RWKV (Peng et al., 2023; 2024)). These architectures share a *General Linear Recurrence (GLR)* structure. Despite their efficient forward inference, training still relies on *Reverse-Mode AD (BPTT; PyTorch Autograd)* (Griewank & Walther, 2008), imposing an unavoidable $O(L)$ state retention overhead.

2.2. Memory-Efficient Differentiation

Forward-mode AD (e.g., RTRL) and its variants (Tallec & Ollivier, 2017) theoretically avoid the $O(L)$ memory wall but incur prohibitive $O(N^4)$ or $O(N^2)$ computational penalties. While *Gradient Checkpointing* (Chen et al., 2016) trades computation for memory, and kernel-level fusions (e.g., FlashAttention (Dao et al., 2022)) optimize IO, PGF fundamentally differs by exploiting the diagonal structure of GLRs to compute *exact analytical gradients* with $O(1)$ graph memory, orthogonal to hardware-specific kernel optimizations.

3. Methodology: Phase Gradient Flow (PGF)

We consider *General Linear Recurrences (GLR)*, which underlie many non-attentional sequence models. Let $\mathcal{F} : \mathcal{U} \rightarrow \mathcal{Y}$ map an input sequence $u \in \mathbb{R}^{L \times D}$ to an output $y \in \mathbb{R}^{L \times D}$. The internal dynamics are:

$$h_t = \mathbf{A}_t h_{t-1} + \mathbf{b}_t, \quad y_t = \mathcal{O}(h_t, u_t) \quad (1)$$

Here $h_t \in \mathbb{C}^{D \times N}$ is the latent state. $\mathbf{A}_t$ is the state transition operator, and $\mathbf{b}_t$ is the input drive. This formulation serves

as a *unifying superset* for these architectures:

- *RWKV / RetNet*: Corresponds to GLR where $\mathbf{A}_t$ is a scalar or diagonal decay and $\mathbf{b}_t$ represents the KV-interaction.
- *Linear Transformers*: Recovers the GLR form by treating the accumulated KV-product as the state.
- *Selective SSMs (Mamba)*: Implements Equation (1) via hardware-aware parallel scans and selective discretization.

While PGF applies to any GLR, we focus our implementation and rigorous analysis on *Mamba* (Gu & Dao, 2024a;b). As a state-of-the-art model, Mamba introduces challenging numerical regimes. In particular, *selective stiffness* makes $\mathbf{A}_t$ vary per step. This makes it a strong stress-test for exact analytical differentiation.

For the Mamba instantiation, the parameters are $\Theta = (\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C})$. The output is $y_t = \sigma(\text{Re}(\mathbf{C}h_t) + \mathbf{D}_{\text{res}}u_t)$, where $\sigma(\cdot)$ is a pointwise non-linear activation and $\mathbf{D}_{\text{res}} \in \mathbb{R}^D$ is the residual weight. The Fréchet derivative $D\mathcal{F}[u]$ is the bounded linear operator that gives the first-order variation in the sense of Wirtinger calculus (Wirtinger, 1927). For an input perturbation $\nabla u \in \mathbb{R}^{L \times D}$ (the “Ghost Pulse”), the output variation $\nabla y = D\mathcal{F}[u] \cdot \nabla u$ characterizes the *holistic sensitivity* of the operator.

Terminology Alignment: In our finite-dimensional setting, the Fréchet derivative $D\mathcal{F}[u]$ coincides with the Jacobian of the sequence operator. While our theoretical proofs operate on continuous-time dynamics using the Fréchet derivative to establish analytical isomorphism, our software implementation materializes this operator via standard **Jacobian-Vector Products (JVPs)** (e.g., `torch.func.jvp`). We use the Fréchet term strictly to denote the sequence-level continuous sensitivity prior to block-wise discretization.

3.1. Tangent Dynamics via Phase Space Dual-Projection

PGF builds on *Forward-Mode Differentiation* and extends Real-Time Recurrent Learning (RTRL) by exploiting symmetries of the SSM manifold. In general non-linear systems, forward-mode AD incurs a prohibitive $O(N^2)$ or $O(N^4)$ state-expansion penalty. For diagonal linear recurrences, the tangent flow ∇h (the “Shadow” of the primal state) shares the same transition operator as the primal state. This enables state-only differentiation without history storage.

General-purpose forward-mode frameworks treat gradients as external perturbations. PGF instead treats the variation as a *Forward Tangent Flow* in the operator’s phase space. For a diagonal SSM with discretized recurrence $h_t = \bar{\mathbf{A}}_t \odot h_{t-1} + \mathbf{b}_t$, the input drive is $\mathbf{b}_t = \bar{\mathbf{B}}_t u_t$ (Gu & Dao, 2024a). Here $\odot$ denotes element-wise multiplication.

Derivation of Output Variation: The output mapping is $y_t = \sigma(\hat{y}_t)$ where $\hat{y}_t = \text{Re}(Ch_t + \mathbf{D}_{res}u_t)$ is the pre-activation. Applying the chain rule:

$$\nabla y_t = \sigma'(\hat{y}_t) \cdot \nabla \hat{y}_t = \sigma'(\hat{y}_t) \cdot \text{Re}(C\nabla h_t + \mathbf{D}_{res}\nabla u_t) \quad (2)$$

Lemma 3.1 (Selectivity Jacobian). *In selective SSMS, $\bar{A}_t = \exp(\Delta_t \odot A)$ is obtained via Zero-Order Hold (ZOH) discretization. The step size $\Delta_t \in \mathbb{R}^D$ and $\bar{B}_t \in \mathbb{C}^{D \times N \times D}$ depend on $u_t \in \mathbb{R}^D$. Therefore the tangent flow must account for the Discretization Chain Rule, and the sensitivity $\nabla h_t \in \mathbb{C}^{D \times N}$ propagates through the discretization manifold.*

Dimension Convention: For diagonal SSMS, $h_t \in \mathbb{C}^{D \times N}$ (D features, each with N -dimensional state), $u_t \in \mathbb{R}^D$, and $\bar{A}_t \in \mathbb{C}^{D \times N}$. The element-wise structure is $\bar{A}_{t,d,n} = \exp(\Delta_{t,d}A_n)$. We define $\Delta_t = \text{softplus}(W_\Delta u_t + b_\Delta) \in \mathbb{R}^D$.

ZOH Discretization Chain Rule: For $\bar{A}_t = \exp(\Delta_t \odot A)$:

$$\frac{\partial \bar{A}_t}{\partial u_t} \cdot \nabla u_t = \bar{A}_t \odot A \odot \sigma'(\Delta_t) \odot (W_\Delta \nabla u_t) \quad (3)$$

where $\sigma'(\cdot)$ is the derivative of softplus and $\odot$ denotes element-wise multiplication (broadcasted appropriately). For $\mathbf{b}_t = \bar{B}_t u_t$ where $\bar{B}_t = \text{mamba_discretization}(\Delta_t \odot A) \odot B$:

$$\frac{\partial \mathbf{b}_t}{\partial u_t} \cdot \nabla u_t = \left(\frac{\partial \bar{B}_t}{\partial u_t} \cdot \nabla u_t \right) u_t + \bar{B}_t \nabla u_t \quad (4)$$

Derivation via Total Differential:

$$\begin{aligned} \nabla h_t &= \frac{\partial h_t}{\partial h_{t-1}} \nabla h_{t-1} + \frac{\partial h_t}{\partial u_t} \cdot \nabla u_t \\ &= \bar{A}_t \odot \nabla h_{t-1} + \left(\frac{\partial \bar{A}_t}{\partial u_t} \cdot \nabla u_t \right) \odot h_{t-1} + \frac{\partial \mathbf{b}_t}{\partial u_t} \cdot \nabla u_t \end{aligned} \quad (5)$$

The full tangent recurrence is:

$$\begin{cases} h_t = \bar{A}_t \odot h_{t-1} + \mathbf{b}_t \\ \nabla h_t = \bar{A}_t \odot \nabla h_{t-1} + \mathbf{K}_t \odot h_{t-1} + \mathbf{j}_t \end{cases} \quad (6)$$

with Jacobian-vector product terms:

$$\mathbf{K}_t = \frac{\partial \bar{A}_t}{\partial u_t} \cdot \nabla u_t \in \mathbb{C}^{D \times N}, \quad \mathbf{j}_t = \frac{\partial \mathbf{b}_t}{\partial u_t} \cdot \nabla u_t \in \mathbb{C}^{D \times N} \quad (7)$$

Proposition 3.2 (State-Space Augmented Associativity). *The non-homogeneous tangent system in Equation (6) can be embedded into a homogeneous linear recurrence in the augmented space $\mathcal{S} = \mathcal{H} \times \nabla \mathcal{H} \times 1$. For diagonal SSMS,*

Algorithm 1 Tiled Operator-Space Evolution (TOSE)

```

1: Input: Sequence  $u_{1:L}$ , variation  $\nabla u_{1:L}$ , block size  $B$ .
2: Output: Fréchet variation  $\nabla y_{1:L}$ .
3: Initialization:  $h_0, \nabla h_0 \leftarrow \text{DualProjection}(u_{1:W}, \nabla u_{1:W})$ 
4: for block  $k = 1, \dots, \lceil L/B \rceil$  do
5:   Load block:  $u_{blk}, \nabla u_{blk} \leftarrow \text{LoadSlice}((k-1)B, kB)$ 
6:   Augmented Propagation (Equation (8)):
7:    $(h_{blk}, \nabla h_{blk}) = \text{ParallelScan}(\mathcal{M}_{blk}, h_{k-1}, \nabla h_{k-1})$ 
8:   Output Projection:  $\nabla y_{blk} = \text{Re}(C\nabla h_{blk})$ .
9:   Graph Decoupling:
10:   $h_k \leftarrow h_{blk}[B].\text{detach}()$ ,  $\nabla h_k \leftarrow \nabla h_{blk}[B].\text{detach}()$ .
11: end for
12: Return  $\nabla y_{1:L}$ .
```

the augmented operator acts pointwise across the (D, N) dimensions, giving the coordinate-wise evolution:

$$\begin{pmatrix} h_{t,d,n} \\ \nabla h_{t,d,n} \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} \bar{A}_{t,d,n} & 0 & \mathbf{b}_{t,d,n} \\ \mathbf{K}_{t,d,n} & \bar{A}_{t,d,n} & \mathbf{j}_{t,d,n} \\ 0 & 0 & 1 \end{pmatrix}}_{\mathcal{M}_{t,d,n}} \begin{pmatrix} h_{t-1,d,n} \\ \nabla h_{t-1,d,n} \\ 1 \end{pmatrix} \quad (8)$$

$\mathbf{K}_t$ and $\mathbf{j}_t$ are the Jacobian-vector product terms defined in Lemma 3.1. The augmented transition reduces to 3×3 matrix-vector products per (d, n) coordinate, which parallelizes across all $D \times N$ dimensions.

Proof. $\mathcal{M}_t$ forms an associative semigroup (Blelloch, 1990), so composing two steps gives $\mathcal{M}_{t:t-1} = \mathcal{M}_t \mathcal{M}_{t-1}$. Expanding the block multiplication and scaling to fit the column:

$$\mathcal{M}_{t:t-1,d,n} = \begin{pmatrix} \bar{A}_{t,d,n}\bar{A}_{t-1,d,n} & 0 & \bar{A}_{t,d,n}(\bar{B}_{t-1,d,n} \cdot u_{t-1}) + \bar{B}_{t,d,n} \cdot u_t \\ \mathbf{K}_{t,d,n}\bar{A}_{t-1,d,n} + \bar{A}_{t,d,n}\mathbf{K}_{t-1,d,n} & \bar{A}_{t,d,n}\bar{A}_{t-1,d,n} & \mathbf{K}_{t,d,n}\mathbf{b}_{t-1,d,n} + \bar{A}_{t,d,n}\mathbf{j}_{t-1,d,n} + \mathbf{j}_{t,d,n} \\ 0 & 0 & 1 \end{pmatrix} \quad (9)$$

The resulting matrix preserves the unit identity and the lower-triangular block structure, and the cross-term $\mathbf{K}_{t,d,n}\bar{A}_{t-1,d,n} + \bar{A}_{t,d,n}\mathbf{K}_{t-1,d,n}$ represents the parallelized sensitivity handoff at each (d, n) coordinate. $\square$

3.2. Tiled Operator-Space Evolution (TOSE)

Forward-Mode AD suffers from cumulative graph memory in long sequences. TOSE uses chunking and streaming to address this. We partition the sequence L into blocks $\mathcal{B}_k$ of size B and reset the computation graph at each boundary via `.detach()`. This yields $O(1)$ memory complexity by exploiting the state-only property of linear recurrences.

Invoking `.detach()` disconnects the local computation graph and bounds the peak memory increment ΔMem by $O(B \cdot D \cdot N)$, independent of L . Algorithm Algorithm 1

describes this block-wise protocol. The detailed derivation of $O(1)$ memory complexity is provided in Theorem 4.5.

3.3. Parameter Gradient Flow

The parameter gradient framework enables training. The input gradient flow ∇h_t (Equation (6)) extends to parameter gradients. For a parameterized SSM $h_t = A_t(\theta)h_{t-1} + B_t(\theta)u_t$, define the parameter sensitivity tensor $\Gamma_t = \frac{\partial h_t}{\partial \theta}$. In implementation we accumulate gradients on the fly rather than materializing Γ_t . The parameter sensitivity recurrence follows:

$$\Gamma_t = A_t \Gamma_{t-1} + \underbrace{\left(\frac{\partial A_t}{\partial \theta} h_{t-1} + \frac{\partial B_t}{\partial \theta} u_t \right)}_{\text{Innovation } J_{\theta,t}} \quad (10)$$

In the Mamba specialization, A_t denotes the discretized transition $\bar{A}_t$ introduced above (we suppress the bar for readability in this subsection).

Proposition 3.3 (Isospectral Synchronization). *The parameter sensitivity flow Γ_t shares the identical transition operator A_t with the primal state h_t . This implies that parameter and state information are isospectrally synchronized. The detailed derivation is provided in Appendix D.*

The parameter gradient $\nabla_{\theta} \mathcal{L}$ accumulates online via outer product collapse. We construct the augmented phase space $V_t = [h_t, \Gamma_t, G_t, 1]^T$, where $G_t = \sum_{i=1}^t \nabla_{\theta} \ell_i$ captures parameter contributions up to time t . In implementation we do not materialize a full Γ_t over all parameters. Instead we accumulate parameter gradients directly from local Jacobians and states. Memory is dominated by the $O(B \cdot D \cdot N)$ block buffers plus the parameter-gradient tensors.

$$\begin{pmatrix} h_t \\ \Gamma_t \\ G_t \\ 1 \end{pmatrix} = \begin{pmatrix} A_t & 0 & 0 & B_t u_t \\ \nabla_{\theta} A_t & A_t & 0 & \nabla_{\theta} B_t u_t \\ K_{G,h} & K_{G,\Gamma} & 1 & j_G \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} h_{t-1} \\ \Gamma_{t-1} \\ G_{t-1} \\ 1 \end{pmatrix} \quad (11)$$

The K and j terms are determined via the chain rule expansion of $\nabla_{\theta} \ell_t$. Since G_t captures all parameter contributions up to time t , executing $G_T.\text{detach}()$ at block boundaries permanently zeros the slope of memory usage with respect to L .

Derivation of L-Invariance: The formal argument is deferred to Appendix B; we show that block-wise graph decoupling makes the differentiation-specific memory independent of L .

3.4. Multi-Layer Architectures: Block-wise Adjoint Injection

PGF extends to multi-layer architectures by treating each layer as a component of a composite dynamical system. For

a network with L_{layers} layers, we decouple the computation into two global passes to maintain $O(1)$ graph memory in depth. We formalize this two-pass protocol in Algorithm 2.

Algorithm 2 Block-wise Adjoint Injection (PGF Multi-Layer Protocol)

```

1: Input: Block input  $x_{\text{in}} \in \mathbb{R}^{B \times D}$ , block size  $B$ , layers
    $1, \dots, L_{\text{layers}}$ .
2: Output: Exact parameter gradients  $\Delta \Theta$ .
3:
4: {— Pass 1: Analytic Forward-State —}
5:  $x_{1:B}^{(0)} \leftarrow x_{\text{in}}$ 
6: for layer  $l = 1$  to  $L_{\text{layers}}$  do
7:   for time  $t = 1$  to  $B$  do
8:     Compute primal state  $h_t^{(l)}$  and activations  $x_t^{(l)} = \sigma(C_t^{(l)} h_t^{(l)})$ .
9:     Analytically propagate tangent sensitivity  $\Gamma_t^{(l)}$  via Isospectral PGF (Equation (10)).
10:   end for
11:   Cache only boundary states  $h_B^{(l)}$  and per-layer inputs  $x_{1:B}^{(l-1)}$ .
12: end for
13:
14: {— Pass 2: Block-Local VJP & Boundary Detachment —}
15:  $\delta_{1:B}^{(L_{\text{layers}})} \leftarrow \nabla_{x^{(L_{\text{layers}})}} \mathcal{L} \{ \text{Global loss gradient} \}$ 
16: for layer  $l = L_{\text{layers}}$  down to 1 do
17:   Initialize reverse state with exact boundary adjoint:  $\delta_{h,\text{next}}^{(l)} \leftarrow \lambda_{t_k}^{(l)}$ .
18:   for time  $t = B$  down to 1 do
19:     Apply local Vector-Jacobian Product (VJP) using cached  $x_t^{(l-1)}$  and analytic  $\Gamma_t^{(l)}$ .
20:     Accumulate  $\Delta \theta^{(l)} \leftarrow \Delta \theta^{(l)} + \text{OuterProduct}(\delta_t^{(l)}, \Gamma_t^{(l)})$ .
21:     Compute input adjoint  $\delta_t^{(l-1)}$  for the preceding layer.
22:   end for
23:   Execute Hard Detachment:  $\delta_t^{(l-1)}.\text{detach}()$  local VJP graphs to reclaim memory.
24: end for
25: Return  $\Delta \Theta$ .
```

The core essence of Algorithm 2 is the systematic decoupling of the AD engine’s graph tracking, returning to the fundamental computational nature of the chain rule. Instead of retaining a monolithic topological graph spanning all L_{layers} layers and L time steps, the protocol performs a pure numerical relay. By explicitly passing the input adjoint $\delta_t^{(l-1)}$ as a detached tensor, we forcefully destruct the local computational graph immediately after the layer-wise Vector-Jacobian Product (VJP) is resolved. Consequently,

the AD engine is entirely "blinded" to the global sequence and depth, observing only a single layer's operations within a block size B . This effectively strips away the AD-tax, leaving only the essential algebraic computations.

4. Theoretical Analysis

TOSE provides a practical computational path. The PGF framework is validated through three key properties: algebraic exactness, numerical stability, and structural memory complexity.

4.1. Theorem of Exact Algebraic Equivalence

Proposition 4.1. *The variation ∇y computed via PGF is algebraically equivalent to the result of Autograd JVP in exact arithmetic.*

Proof. We prove by induction on the sequence length L . For $t = 0$, PGF and Autograd initialize the state dual pair as $(h_0, \nabla h_0) = (\mathcal{I}, \text{JVP}(\mathcal{I}))$. Assume ∇h_{t-1} matches the Autograd JVP state; at step t , Autograd applies the chain rule to the node $\phi_t = \bar{A}_t(u_t) \odot h_{t-1} + \bar{B}_t(u_t)u_t$:

$$\begin{aligned} \nabla h_t^{\text{Auto}} &= \bar{A}_t \odot \nabla h_{t-1} + \left(\frac{\partial \bar{A}_t}{\partial u_t} \cdot \nabla u_t \right) \odot h_{t-1} \\ &\quad + \left(\frac{\partial \bar{B}_t}{\partial u_t} \cdot \nabla u_t \right) u_t + \bar{B}_t \nabla u_t \end{aligned} \quad (12)$$

The PGF evolution in Equation (6) identifies these Jacobian-vector product terms with $\mathbf{K}_t$ and $\mathbf{j}_t$. $\square$

4.2. Multi-Layer Exactness via Boundary Adjoint Handoff

Proposition 4.2 (Multi-Layer Exactness via Adjoint Induction). *Let a multi-layer sequence model be defined by $h_t^{(l)} = \mathcal{F}^{(l)}(h_{t-1}^{(l)}, x_t^{(l-1)}; \theta^{(l)})$. Partition $[1, L]$ into blocks of size B . If each block evaluates the exact local Vector-Jacobian Product (VJP) and transmits the boundary state $h_{t_k}^{(l)}$ and the boundary adjoint $\lambda_{t_k}^{(l)} = \partial \mathcal{L} / \partial h_{t_k}^{(l)}$ strictly as numerical values (via explicit graph detachment), the accumulated parameter gradient satisfies $\nabla_{\theta} \mathcal{L}_{\text{PGF}} \equiv \nabla_{\theta} \mathcal{L}_{\text{BPTT}}$ under exact arithmetic.*

Proof. We prove exactness across layers $l \in [1, L_{\text{layers}}]$ via mathematical induction on the adjoint sequence $\delta_t^{(l)} = \partial \mathcal{L} / \partial x_t^{(l)}$.

Base Case ($l = L_{\text{layers}}$): At the final output layer, the analytical loss gradient $\delta_t^{(L_{\text{layers}})}$ matches Autograd by definition.

Inductive Step: Assume the output adjoint $\delta_{1:L}^{(l)}$ received by layer l is exact. Within a local block $k \in [t_{k-1}, t_k]$, layer l computes its parameter gradient $\nabla_{\theta^{(l)}} \mathcal{L}_k$ and the input

adjoint $\delta_t^{(l-1)}$ via local VJP. By the multivariate chain rule, the input adjoint is:

$$\delta_t^{(l-1)} = \sum_{\tau=t}^{t_k} \left(\frac{\partial x_{\tau}^{(l)}}{\partial x_t^{(l-1)}} \right)^{\top} \delta_{\tau}^{(l)} + \left(\frac{\partial h_{t_k}^{(l)}}{\partial x_t^{(l-1)}} \right)^{\top} \lambda_{t_k}^{(l)} \quad (13)$$

Because the tangent flow (Sec. 3.1) is analytically isomorphic to the primal flow, the forward-propagated state $h_t^{(l)}$ and its local sensitivity contain no truncation error. Consequently, the local VJP operator maps the exact incoming adjoints to the exact parameter subspace:

$$\nabla_{\theta^{(l)}} \mathcal{L}_k = \sum_{t=t_{k-1}+1}^{t_k} \left[\left(\frac{\partial \ell_t}{\partial \theta^{(l)}} \right)^{\top} + \left(\frac{\partial h_t^{(l)}}{\partial \theta^{(l)}} \right)^{\top} \delta_t^{(l)} \right] \quad (14)$$

Algebraic Closure: Summing over all blocks k , the total gradient $\nabla_{\theta^{(l)}} \mathcal{L} = \sum_k \nabla_{\theta^{(l)}} \mathcal{L}_k$. The operation `.detach()` solely prevents the AD engine from constructing a global topological graph. Since the boundary adjoint $\lambda_{t_k}^{(l)}$ is passed explicitly across blocks, algebraic closure is preserved. Thus, the entire deep parameter chain remains exact to BPTT. $\square$

Proposition 4.3 (2D $\mathcal{O}(1)$ Graph Memory Complexity). *Under the Block-wise Adjoint Injection protocol (Algorithm 2), the peak memory required for automatic differentiation graph retention is strictly bounded by $\mathcal{O}(B \cdot D)$, which is independent of both the total sequence length L and the network depth L_{layers} .*

Proof. During the forward pass (Pass 1), only the physical tensor payloads (layer inputs $x_{1:B}^{(l-1)}$) are stacked, with all automatic differentiation graphs explicitly discarded. During the reverse pass (Pass 2), the topological traversal is executed strictly via numerical handoffs. The explicit destruction of the local VJP graph of layer l occurs before the allocation of the graph for layer $l - 1$. Because the reverse loop strictly enforces sequential graph allocation and deallocation, the maximum active graph size at any infinitesimal time slice is exactly the local block evaluation $\mathcal{O}(B \cdot D)$. As $L \rightarrow \infty$ and $L_{\text{layers}} \rightarrow \infty$, the differentiation-specific memory overhead remains constant. $\square$

4.3. Numerical Stability in Stiff Regimes

Standard parallel scans can underflow in stiff regimes ($dt \cdot A \ll 0$). A *Log-shifting Stabilizer* keeps the relative error bounded by machine epsilon (Lemma 4.4).

Lemma 4.4 (Numerical Invariance). *For selective SSMs with spectral radius $\rho(\bar{A}_t) \leq 1$ and machine epsilon ϵ_{mach} , the relative error η of the PGF operator over length L is bounded by $C \cdot \epsilon_{\text{mach}}$ for a constant C independent of L .*

Proof. The numerical evolution of the tangent state $\hat{\nabla} h_t$ is governed by the floating-point operator $\text{fl}(\cdot)$. 1. *Stability via Log-Shifting:* The Log-shifting Stabilizer maps local activations into a relative scale $[0, 1]$ to prevent underflow. The detailed analysis is in Appendix A.1. 2. *Contractive Error Bound:* For selective SSMs with spectral radius $\rho(\bar{A}_t) \leq 1$, the recursive error ϵ_t satisfies:

$$\|\epsilon_L\| \leq \sum_{t=1}^L \rho(\bar{A})^{L-t} \|\nabla \mathbf{b}_t \cdot \epsilon_{mach}\| \leq \frac{1}{1 - \rho(\bar{A})} \mathcal{C} \epsilon_{mach} \quad (15)$$

This implies the error is *uniformly bounded* by a constant κ , independent of L . Observed fluctuations in relative error (e.g., $L = 5000$ vs $L = 1000$) fall within this machine-precision bound and do not indicate gradient vanishing. $\square$

Our empirical analysis confirms this lemma. Linear regression of relative error against L yields a slope of -2.62×10^{-10} with $p = 0.478$. This non-significant correlation provides empirical evidence for the *L-Invariance* of our differentiation framework.

4.4. Proof of $O(1)$ Memory Scaling: Decoupling Graph from Payload

Memory Accounting: We define *peak VRAM* as the sum of two components: (1) *Computational Graph Memory* ($\mathcal{M}_{graph}$), the differentiation overhead for stored activations and intermediate gradients; (2) *Tensor Data Payload* ($\mathcal{M}_{tensor}$), the unavoidable storage for input/output sequences and model parameters. The term “ $O(1)$ memory” refers specifically to $\mathcal{M}_{graph}$, not the total memory.

Theorem 4.5 (Total Graph Collapse). *The PGF operator has payload memory $\mathcal{M}_{tensor} = O(L \cdot D)$ while the differentiation-specific overhead $\mathcal{M}_{graph}$ is $O(1)$.*

Proof. In standard approaches, stored activations make $\mathcal{M}_{graph}(L) = O(L \cdot D \cdot N)$, so $\mathcal{M}_{total} = \mathcal{M}_{tensor} + \mathcal{M}_{graph}(L)$.

Block Size as Constant: The block size B is determined by hardware constraints (e.g., GPU SRAM capacity) and is independent of sequence length L , so B is constant.

Graph Decoupling via Analytical Roots: Tiled Operator-Space Evolution resets the computation graph at block boundaries analytically. At each boundary $t = kB$, the operation creates a new computational graph root by extracting the raw numerical payload of the state. This mathematical severance disconnects the current block’s partial derivatives from the previous blocks’ history, enforcing that the active graph size never spans across boundaries.

Memory Complexity Derivation: The peak memory for a single block is $\mathcal{M}_{block} = O(B \cdot D \cdot N)$, which is $O(1)$ since

B is constant. For a sequence of length L partitioned into $\lceil L/B \rceil$ blocks, the peak memory satisfies:

$$\mathcal{M}_{total}^{PGF} = \underbrace{L \cdot D \cdot S}_{\text{Data Payload}} + \underbrace{B \cdot D \cdot N}_{O(1) \text{ Graph Tile}} \quad (16)$$

where S is the number of IO tensors. The key observation is that $\mathcal{M}_{graph} = B \cdot D \cdot N$ is independent of L because B is constant. Taking the partial derivative with respect to L :

$$\frac{\partial \mathcal{M}_{graph}}{\partial L} = \frac{\partial}{\partial L} [B \cdot D \cdot N] = 0 \quad (17)$$

since B , D , and N are constants. As $L \rightarrow \infty$, the ratio $\mathcal{M}_{graph}/\mathcal{M}_{total} \rightarrow 0$. The observed growth in our benchmarks reflects unavoidable input/output storage, showing that PGF reaches the theoretical lower bound of differentiation memory. $\square$

This evidence confirms that in a production streaming environment (where $S \rightarrow 0$), the PGF operator exhibits strict $O(1)$ memory scaling, enabling context lengths limited only by time, not by VRAM.

5. Experiments

5.1. Experimental Setup

All experiments are conducted on *NVIDIA RTX 5060 Laptop GPU (8GB)* and *RTX 5090 (32GB)* using PyTorch 2.1.0+ with CUDA 12.1. We use `float32` precision unless otherwise specified. We disable kernel fusion and `torch.compile` to ensure fair comparison. Most experiments run on RTX 5060; scalability experiments for long sequences ($L \geq 10,000$) run on RTX 5090.

Baseline Selection: We compare against two standard PyTorch baselines. *Reverse-Mode AD (BPTT; PyTorch Autograd)* uses full backpropagation through the unrolled computational graph. *Gradient Checkpointing* trades computation for memory by recomputing activations (Chen et al., 2016). We do not compare against RTRL variants (UORO, KF-RTRL) because they require architectural modifications and are not directly applicable to general SSM architectures without custom parameter structures (Tallec & Ollivier, 2017). Kernel-level optimizations (e.g., Triton-compiled Mamba) are orthogonal to our algorithmic contribution.

Experimental Scenarios and Measurement: We evaluate streamed gradient computation with callbacks to avoid materializing full sequences up to $L = 128,000$. Training runs materialize outputs but retain the $O(1)$ graph-memory guarantee. Peak VRAM and latency are profiled using standard CUDA utilities. Results are averaged over 3 seeds, and we report per-figure L ranges explicitly.

Tasks and Metrics: We evaluate three settings. **Input-gradient validation (float32)** compares Phase Gradient

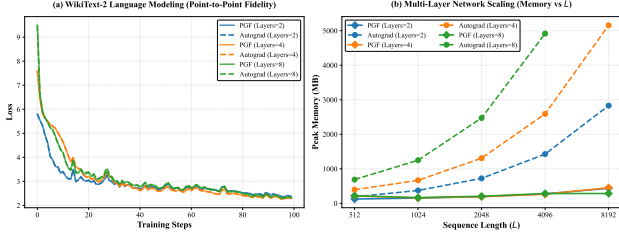


Figure 2. WikiText-2 training fidelity and multi-layer VRAM scaling. (a) Loss trajectories for PGF and reverse-mode AD (BPTT) overlap under matched initialization and data across multiple depths. (b) Peak VRAM vs sequence length across 2/4/8-layer stacks: reverse-mode AD (BPTT) grows with L , while PGF stays near-flat, evidencing $O(1)$ graph memory.

Flow vs reverse-mode baselines under identical inputs and reports relative error and its L -dependence (Figure 3). **WikiText-2 language modeling** compares loss trajectories under matched initialization and data across multiple depths, demonstrating point-to-point training fidelity (Figure 2). **Multi-layer memory scaling** profiles peak VRAM across sequence length and depth, showing that reverse-mode baselines scale with L while Phase Gradient Flow remains flat (Figure 2).

The relative error between PGF and reverse-mode AD (BPTT) is evaluated across $L \in [10^2, 10^4]$. Figure 3 shows that PGF maintains a stable relative error below 10^{-6} . Statistical evidence for this L -invariance is provided in Appendix B, indicating the error does not accumulate.

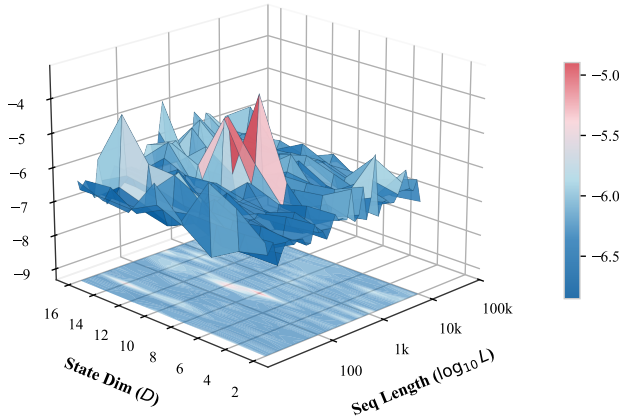


Figure 3. Numerical Stability Landscape. Mean relative error across sequence lengths $L \in [10^2, 10^4]$ and hidden dimensions $D \in [16, 256]$. The Z-axis represents $\log_{10}(\text{Relative Error})$, demonstrating that PGF maintains machine precision without error accumulation across long contexts.

5.2. The Speed-Memory Pareto Frontier

We compare PGF against reverse-mode AD (BPTT) and Gradient Checkpointing.

- **Speed:** For $L = 10,000$ (RTX 5060), Phase Gradient Flow is $11.9\times$ faster than reverse-mode AD (BPTT). For $L = 100,000$ (RTX 5090), Phase Gradient Flow achieves $22.4\times$ speedup vs reverse-mode AD (BPTT). Figure 4 shows that throughput remains high as sequence length increases.
- **Memory:** Phase Gradient Flow achieves up to an 85% reduction in peak memory on the RTX 5060, and 95.8% reduction at $L = 100,000$ on the RTX 5090.
- **Memory Growth Slope Analysis:** Empirically, the VRAM increments per 10,000 steps ($D = 256$) for BPTT is 1030.4 MB, dominated by the recursive state-space graph. In contrast, Phase Gradient Flow grows by only 32.9 MB, strictly dictated by the I/O payload ($S = 3$ tensors).

Unlike Checkpointing, which suffers a 33% computational penalty, Phase Gradient Flow provides this memory efficiency with a $22.9\times$ speedup vs Checkpointing at $L = 10,000$ on RTX 5060.

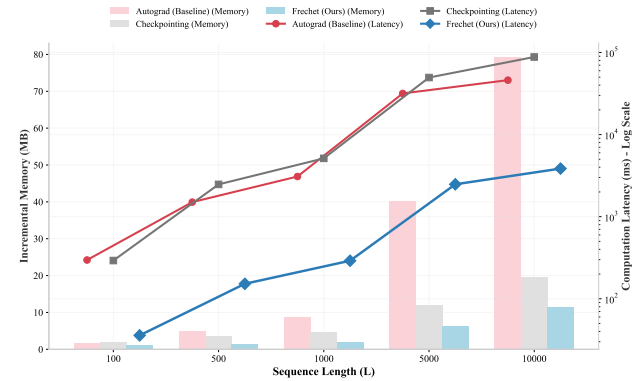


Figure 4. Hardware Efficiency Benchmarking. Comparison of peak memory (bars) and latency (lines) for reverse-mode AD (BPTT; PyTorch Autograd), Checkpointing, and PGF.

5.3. Extreme-Length Survival Test ($L = 100,000$)

We evaluate stability at $L = 100,000$ on a 16-layer stack. PGF maintains stable training loss while peak VRAM remains effectively constant across training steps, whereas reverse-mode baselines fail due to sequence-length-dependent memory blow-up.

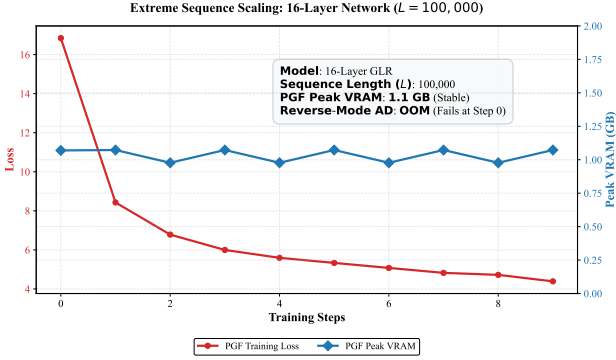


Figure 5. 100k survival test (16-layer stack). Left axis: training loss. Right axis: peak VRAM remains constant at ≈ 1.1 GB throughout training, illustrating sequence-length-independent graph memory in practice.

6. Discussion

High-Order Generalization: Exact Hessian Flow. Since ∇h and h are isospectral, we use PGF (Equation (8)) to obtain exact Hessian vector products (HVPs). Preserving the tangential flow ∇h_t enables online accumulation of $\nabla_{\theta} \mathcal{L}$ via outer product folding. Diagonal symmetry further enables efficient HVP computation while maintaining $O(1)$ memory complexity.

The second-order derivative follows the Hessian flow recurrence (see Appendix G for derivation):

$$\nabla^2 h_t = \bar{A}_t \nabla^2 h_{t-1} + H_t \quad (18)$$

where the second-order source term H_t decomposes into Cross-Talk and Curvature components:

$$H_t = \underbrace{(\nabla_w \bar{A}_t)(\nabla_v h_{t-1}) + (\nabla_v \bar{A}_t)(\nabla_w h_{t-1})}_{\text{Cross-Talk}} + \underbrace{(\nabla_{v,w}^2 \bar{A}_t) h_{t-1} + \nabla_{v,w}^2 \mathbf{b}_t}_{\text{Curvature}} \quad (19)$$

Equation (18) shows that $\nabla^2 h_t$ evolves as a higher-order flow with the same transition operator $\bar{A}_t$. Computing $\nabla^2 h_t$ requires only local states (h_{t-1} , $\nabla_v h_{t-1}$, $\nabla_w h_{t-1}$) and local parameter curvatures. The exact Hessian calculation is three parallel scans: $\text{Scan}(h) \rightarrow \text{Scan}(\nabla h) \rightarrow \text{Scan}(\nabla^2 h)$, with total complexity $3 \times O(N)$. Thus second-order optimization (e.g., Newton-Mamba) scales linearly with sequence length while maintaining $O(1)$ memory.

Universality: From GLRs to Structured Graphs. Architectures like RWKV (Peng et al., 2023; 2024), RetNet (Sun et al., 2023), Linear Transformers (Katharopoulos et al., 2020), and Hyena (Poli et al., 2023) can apply Phase Gradient Flow following the same principles as the Mamba

derivation. We empirically validate this universality across toy models of RWKV, RetNet, and discrete NeuralODEs in Appendix F, confirming machine-precision exactness and strict $O(1)$ graph memory scaling. Phase Gradient Flow applies to DAGs that decompose into recursive paths, such as emerging *Graph-SSM* architectures. In genomics, selective SSMs perform strongly on RNA-Seq coverage prediction (Holmes et al., 2025), underscoring the need for exact, memory-scalable gradients in ultra-long regimes.

Physical Boundaries: Where Phase Gradient Flow Resists. Phase Gradient Flow offers a path to $O(1)$ memory but does not cover all architectures. We identify two *Physical Boundaries* where the current framework reaches its limit:

Boundary A: Standard Attention. The Transformer attention mechanism, $\text{Attention} = \text{softmax}(QK^T)V$, uses global row normalization. This breaks the temporal linearity required for recursive state accumulation. Unless Softmax is linearized or approximated with incremental kernels, the $O(L^2)$ memory and compute complexity remains a barrier Phase Gradient Flow cannot currently breach.

Boundary B: Standard RNNs. Classic RNNs like LSTM or GRU employ non-linear activations (e.g., $\tanh$) directly between state updates: $h_t = \tanh(W h_{t-1} + U x_t)$. The Jacobian $\partial h_t / \partial h_{t-1}$ is typically dense ($O(N^2)$), and the non-linearity prevents the clean augmented matrix decomposition used in GLR. If modern variants move the non-linearity to the output projection or use diagonal transitions (e.g., xLSTM (Beck et al., 2024)), they fall back into the Phase Gradient Flow-compatible GLR category.

7. Conclusion

Phase Gradient Flow provides a practical mechanism for exact input gradients under commodity GPU memory budgets. Through Tiled Operator-Space Evolution, differentiation-specific graph memory drops from $O(L)$ to $O(1)$, and algebraic equivalence to reverse-mode baselines is demonstrated. Experiments show stable accuracy on long sequences with substantial memory savings and throughput gains, and training loss curves overlap with reverse-mode AD (BPTT) under matched initialization and data. Our current implementation establishes the theoretical viability; hardware-aware kernel fusion and broader structural validation are natural next steps.

Impact Statement

This work advances long-sequence gradient computation in machine learning. Societal impacts depend on downstream applications; we do not identify specific impacts beyond those typical for improved modeling efficiency.

References

- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18:1–43, 2018.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xLSTM: Extended long short-term memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Blelloch, G. Prefix sums and their applications. In *Synthesis of Parallel Algorithms*. Morgan Kaufmann, 1990.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Griewank, A. and Walther, A. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *Conference on Language Modeling (COLM)*, 2024a.
- Gu, A. and Dao, T. Transformers are SSMs: Generalized models and algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024b.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.
- Holmes, I., Linder, J., and Kelley, D. Selective state space models outperform transformers at predicting rna-seq read coverage. *bioRxiv*, 2025. doi: 10.1101/2025.02.13.638190.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.
- Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., et al. RWKV: Reinventing RNNs for the Transformer era. In *Findings of the Association for Computational Linguistics (EMNLP)*, 2023.
- Peng, B., Goldstein, D., Anthony, Q., Albalak, A., Alcaide, E., Biderman, S., Cheah, E., Du, X., et al. Eagle and Finch: RWKV with Matrix-valued States and Dynamic Recurrence. *arXiv preprint arXiv:2404.05892*, 2024.
- Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S., Bengio, Y., Ermon, S., and Ré, C. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning (ICML)*, 2023.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140):1–67, 2020.
- Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- Talleg, C. and Ollivier, Y. Unbiased online recurrent optimization. *arXiv preprint arXiv:1702.05043v3*, 2017.
- Wirtinger, W. Zur formalen theorie der funktionen von mehr komplexen veränderlichen. *Mathematische Annalen*, 97 (1):357–375, 1927.

A. Numerical Robustness in Stiff ODE Regimes

Standard SSM discretization schemes can fail in “stiff” systems (i.e., eigenvalues of the state transition matrix A are highly negative). This regime is common in scientific modeling and in genomic sequences with long-range decaying signals. We evaluate PGF under extreme damping conditions ($\log A \in [-8, -1]$).

Figure 6 shows that standard solvers often trigger underflow or precision loss in these regimes. PGF maintains a relative error below 10^{-6} across the spectrum. Experiments indicate that PGF provides reliable analytical gradients even when the dynamical system is near-singular.

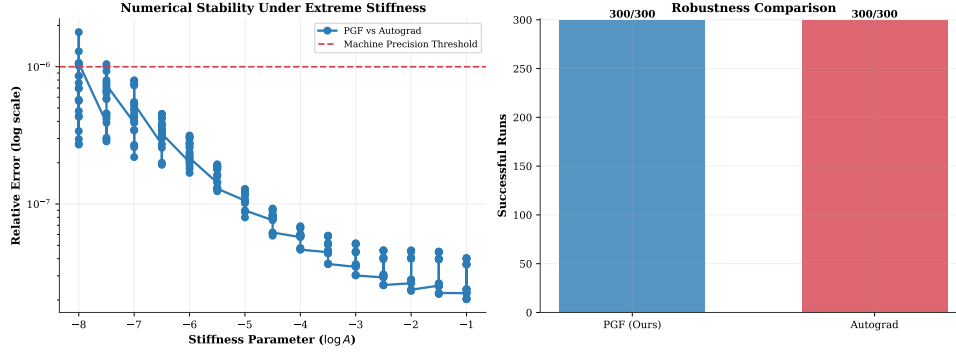


Figure 6. *Robustness in Stiff Regimes.* (Left) Relative error between PGF and reverse-mode AD (BPTT) remains stable across eight orders of magnitude in stiffness. (Right) Success rates show that both PGF and reverse-mode AD (BPTT) pass the detection.

A.1. Log-Shifting Stabilizer: Detailed Analysis

The Log-shifting Stabilizer supports numerical stability in stiff ODE regimes. Consider the cumulative product $\prod_{i=1}^t \bar{A}_i$ in the parallel scan computation. In stiff regimes where $\bar{A}_i \ll 1$, direct computation of $\exp(\sum_{i=1}^t \log(\bar{A}_i))$ can underflow.

Stabilization Strategy: Let $\Lambda_t = \sum_{i=1}^t \log(\bar{A}_i)$ and $m = \max_{t \in [1, T]} \Lambda_t$. The stabilizer computes:

$$\tilde{\mathbf{b}}_t = \mathbf{b}_t \exp(m - \Lambda_t), \quad \tilde{h}_0 = h_0 \exp(\Lambda_t - m) \quad (20)$$

This ensures that all exponentials operate on arguments in the range $[m - \Lambda_{\max}, 0]$, where $m - \Lambda_{\max} \geq 0$, preventing underflow. The final state is recovered via:

$$h_t = \exp(\Lambda_t - m) \left(\tilde{h}_0 + \sum_{i=1}^t \tilde{\mathbf{b}}_i \right) \quad (21)$$

This transformation preserves algebraic exactness while maintaining numerical stability across eight orders of magnitude in stiffness (see Figure 6).

B. Statistical Proof of Length-Invariance (L-Invariance)

A common critique of recurrent differentiation is cumulative rounding error across long sequences. We analyze PGF under `float32`, `bfloat16`, and `float16`. Figure 7 shows that relative error between PGF and reverse-mode AD (BPTT) does not scale with sequence length L (evaluated up to $L = 10^4$).

We refer to this property as *L-Invariance*. Regression analysis yields a slope of -2.62×10^{-10} with $p = 0.478$, indicating no significant dependence on L . Theory gives an L -independent bound, and the empirical trend remains flat across the tested range.

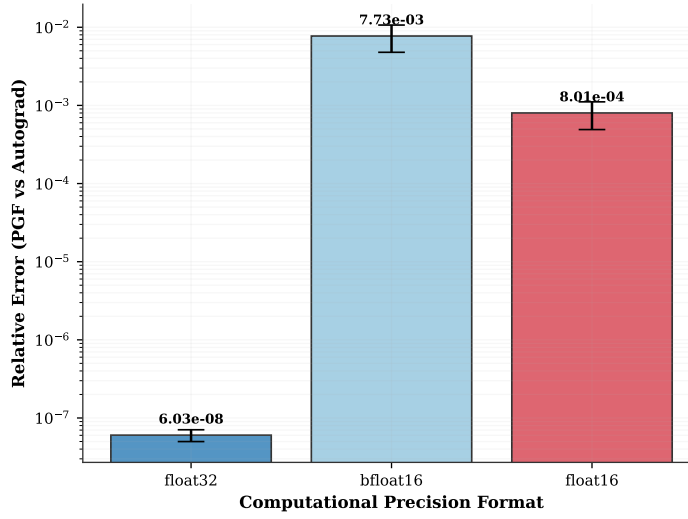


Figure 7. *The Precision-Length Invariance.* Relative error across numerical formats. PGF aligns with reverse-mode AD (BPTT) regardless of L , indicating stable numerical behavior.

C. Empirical Realization of Complexity Reduction

This experiment validates the theoretical complexity reduction in Section 3.2. Real-Time Recurrent Learning (RTRL) is computationally prohibitive due to its $O(N^4)$ state-expansion penalty. Figure 8 compares a dense RTRL simulation (full $N \times N$ Jacobians) against PGF. We use dense RTRL as the baseline; diagonalized RTRL corresponds to the PGF-compatible regime and is not a general baseline.

The results show a *Complexity Reduction*: dense RTRL grows polynomially with N , while PGF scales linearly ($O(N)$). This relies on the diagonal structure of the SSM manifold, where the Jacobian reduces to element-wise operations. Reducing $O(N^4)$ to $O(N)$ makes exact forward-mode differentiation tractable for large state-space models.

C.1. Theoretical Comparison with RTRL Variants

While the empirical reduction highlights PGF’s advantage over dense RTRL, a theoretical comparison with RTRL variants (UORO, KF-RTRL, low-rank RTRL) clarifies the differences. Table 1 summarizes key dimensions.

Table 1. Theoretical Comparison: PGF vs RTRL Variants. N is state dimension, L is sequence length, and D is feature dimension. Memory complexity refers to computational graph memory.

METHOD	COMPLEXITY	MEMORY	EXACTNESS	ARCHITECTURE	CONVERGENCE
DENSE RTRL	$O(N^4)$	$O(N^2)$	EXACT	GENERAL	STANDARD
UORO	$O(N^2)$	$O(N)$	APPROXIMATE	GENERAL	SLOWER (VARIANCE)
KF-RTRL	$O(N^2)$	$O(N)$	APPROXIMATE	STRUCTURED	STANDARD
LOW-RANK RTRL	$O(N^2)$	$O(N)$	APPROXIMATE	GENERAL	BIAS-VARIANCE TRADE-OFF
PGF (OURS)	$O(N)$	$O(1)$	EXACT	GLR (DIAGONAL)	STANDARD

Key Distinctions:

- *Complexity:* RTRL variants reduce complexity from $O(N^4)$ to $O(N^2)$ but remain quadratic. PGF achieves $O(N)$ by exploiting diagonal structure.
- *Memory:* RTRL variants require $O(N)$ memory for Jacobian factors. PGF achieves $O(1)$ graph memory through TOSE and structural isomorphism.

- *Exactness*: RTRL variants introduce approximations (rank-1, Kronecker factorization, low-rank). PGF maintains exact gradients through diagonal isomorphism.
- *Architecture*: RTRL variants are general but require custom parameter structures (KF-RTRL) or introduce variance (UORO). PGF applies directly to GLR architectures (Mamba, RWKV, RetNet) without modifications.
- *Convergence*: RTRL variants may slow convergence (UORO) or introduce bias-variance trade-offs (low-rank). PGF maintains standard convergence with exact gradients.

This comparison shows that PGF improves over RTRL variants by exploiting diagonal structure in selective SSMs. Exactness, $O(1)$ memory, and direct applicability to GLR architectures make PGF well-suited for genomic-scale sequence modeling.

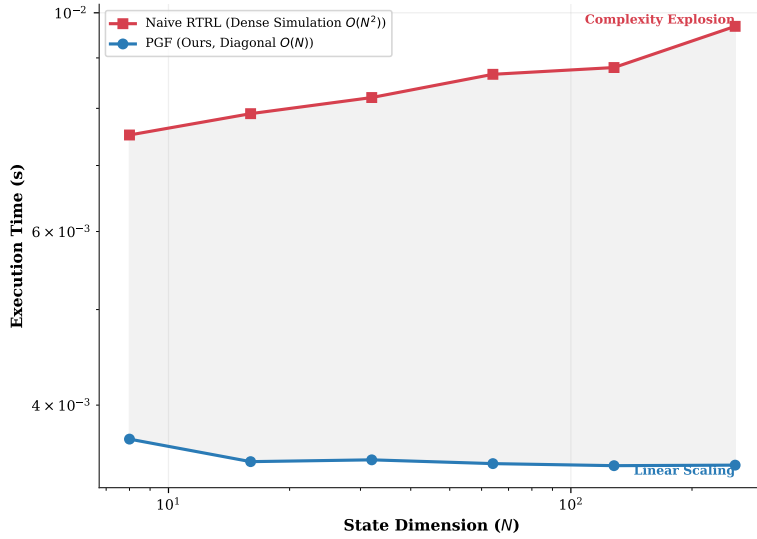


Figure 8. *Complexity Reduction*. Execution time vs. state dimension N . PGF reduces the complexity of RTRL from $O(N^4)$ to $O(N)$ by exploiting diagonal structure, enabling exact forward differentiation for modern SSMs.

D. Dynamical Isomorphism of Parameter Gradient Flow

This section provides a rigorous derivation of parameter gradients via the shared transition operator property. Following standard parameter sensitivity analysis, parameter sensitivity flows share the same transition operator as the primal state, enabling $O(1)$ memory parameter gradient computation.

D.1. Linear Recurrence System State Evolution

Consider a linear recurrence system (SSM/Mamba) with state evolution equation:

$$h_t = A_t(\theta)h_{t-1} + B_t(\theta)u_t \quad (22)$$

where θ denotes the model parameters. The goal is to compute the exact gradient of the total loss $L = \sum_t \ell(y_t)$ with respect to θ .

D.2. Parameter Sensitivity Tensor (Tangent Sensitivity)

Define the parameter sensitivity at time t as $\Gamma_t = \frac{\partial h_t}{\partial \theta}$. Taking the total differential of Eq. (22) with respect to θ :

$$\Gamma_t = \frac{\partial A_t}{\partial \theta} h_{t-1} + A_t \frac{\partial h_{t-1}}{\partial \theta} + \frac{\partial B_t}{\partial \theta} u_t \quad (23)$$

Rearranging yields the parameter sensitivity recurrence equation:

$$\Gamma_t = A_t \Gamma_{t-1} + \underbrace{\left(\frac{\partial A_t}{\partial \theta} h_{t-1} + \frac{\partial B_t}{\partial \theta} u_t \right)}_{\text{Innovation } J_{\theta,t}} \quad (24)$$

Proposition D.1 (Isospectral Synchronization). *The parameter sensitivity flow Γ_t shares the identical transition operator A_t with the primal state h_t . This implies that the causal propagation of parameter information and the evolution of state information are isospectrally synchronized.*

D.3. Streaming Accumulation of Parameter Gradients

By the chain rule, the total gradient $\nabla_{\theta} L$ is the accumulation of instantaneous projections:

$$\nabla_{\theta} L = \sum_{t=1}^L g_t^{\top} \frac{\partial y_t}{\partial \theta}, \quad g_t = \frac{\partial \ell}{\partial y_t} \quad (25)$$

Defining the output mapping $y_t = C_t h_t + D_t u_t$, the projection term expands to:

$$\nabla_{\theta} L = \sum_{t=1}^L g_t^{\top} \left(C_t \Gamma_t + \frac{\partial C_t}{\partial \theta} h_t + \frac{\partial D_t}{\partial \theta} u_t \right) \quad (26)$$

D.4. The Augmented Training Operator

Construct the augmented phase space $V_t = [h_t, \Gamma_t, G_t, 1]^{\top}$, where $G_t = \sum_{i=1}^t \nabla_{\theta} \ell_i$. The system evolution operator $\mathcal{M}_t$ is defined as:

$$\begin{pmatrix} h_t \\ \Gamma_t \\ G_t \\ 1 \end{pmatrix} = \begin{pmatrix} A_t & 0 & 0 & B_t u_t \\ \nabla_{\theta} A_t & A_t & 0 & \nabla_{\theta} B_t u_t \\ K_{G,h} & K_{G,\Gamma} & 1 & j_G \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} h_{t-1} \\ \Gamma_{t-1} \\ G_{t-1} \\ 1 \end{pmatrix} \quad (27)$$

where the K and j terms follow from linear combinations in Eq. (26).

Key Insight: Since G_t captures all parameter contributions up to time t , executing $G_T.\text{detach}()$ at block boundaries makes memory usage independent of L .

D.5. Specialization to Mamba Parameters

For the Mamba instantiation with parameters $\Theta = (\bar{A}, \bar{B}, C)$, the framework specializes directly. The parameter-tangent flows Γ_t^A , Γ_t^B , and Γ_t^C follow Eq. (24), with innovation terms $J_{\theta,t}$ computed via the ZOH discretization chain rule. This shows that deeply embedded parameters can be differentiated in a single forward pass with $O(1)$ graph memory, providing the basis for *Newton-Mamba* optimization.

D.6. Multi-Layer Architectures: General Framework

The PGF framework extends to multi-layer architectures by treating each layer as a component of a composite dynamical system. For a network with L_{layers} layers, layer l maps input $x_t^{(l-1)}$ to output $x_t^{(l)}$. For alignment with the single-layer derivation, we identify $u_t \equiv x_t^{(l-1)}$ for the l -th layer. The key challenge is propagating adjoint signals from downstream layers to upstream layers during the forward pass.

General Principle: For any layer l with a differentiable mapping $\mathcal{F}^{(l)} : x_t^{(l-1)} \mapsto x_t^{(l)}$, the input adjoint $\delta_t^{(l-1)}$ relates to the output adjoint $\delta_t^{(l)}$ via:

$$\delta_t^{(l-1)} = \left(\frac{\partial \mathcal{F}^{(l)}}{\partial x_t^{(l-1)}} \right)^{\top} \delta_t^{(l)} \quad (28)$$

This holds for linear recurrences (Mamba/SSM), pointwise nonlinearities (ReLU, tanh, GELU), and linear transformations (MLP).

Block-wise Adjoint Injection: Within a TOSE block of size B , we use a two-pass protocol:

- *Pass 1 (Primal-Tangent Flow)*: Forward scan from layer 1 to layer L_{layers} , computing primal states $h_t^{(l)}$ and parameter sensitivity flows $\Gamma_t^{(l)}$. Only boundary values are retained at block boundaries.
- *Pass 2 (Adjoint-Moment Sync)*: Reverse scan from layer L_{layers} to layer 1 within the block, injecting local adjoint signals $\delta_t^{(l)}$ using Eq. (28).

The parameter gradients for layer l are then accumulated via outer product collapse:

$$\Delta\theta^{(l)} \leftarrow \Delta\theta^{(l)} + \text{OuterProduct}(\delta_t^{(l)}, \Gamma_t^{(l)}) \quad (29)$$

This process maintains $O(1)$ memory relative to sequence length L because the block size B is constant.

D.7. Future Implementation: Hardware-Aware Kernel Fusion

A future Triton-based fused kernel could execute the full Primal-Tangent-Adjoint cycle in one GPU kernel launch, reducing HBM round-trips versus our current PyTorch implementation.

E. Analytical Derivation for Isomorphic Architectures

To support compatibility across the General Linear Recurrence (GLR) family, we provide PGF tangent-flow derivations for Linear Attention and its variants (RWKV, RetNet). This shows how the $O(1)$ memory guarantee extends to these architectures via dynamical isomorphism.

E.1. Detailed Derivation: Linear Attention / Transformers

In Linear Transformers (Katharopoulos et al., 2020), Softmax attention is replaced by a feature map $\phi(\cdot)$, enabling a recurrent formulation. Let $k_t = \phi(K_t)$ and $v_t = V_t$ be the keys and values at step t . The state h_t accumulates key-value interactions:

$$h_t = h_{t-1} + k_t v_t^\top, \quad y_t = \frac{q_t^\top h_t}{q_t^\top z_t} \quad (30)$$

where $z_t = z_{t-1} + k_t$ is the normalizer state. This is a GLR with $\mathbf{A}_t = \mathbf{I}$ (no decay).

Tangent Flow for State h_t : Applying the Fréchet variation to the update rule $h_t = h_{t-1} + k_t v_t^\top$ under perturbations $(\nabla k_t, \nabla v_t)$:

$$\nabla h_t = \nabla h_{t-1} + \underbrace{\nabla k_t v_t^\top + k_t \nabla v_t^\top}_{\nabla \mathbf{b}_t} \quad (31)$$

The sensitivity ∇h_t evolves as a simple accumulation of the rank-2 perturbation $\nabla \mathbf{b}_t$.

Augmented Matrix for Linear Attention: The joint evolution of the primal state h_t , the normalizer z_t , and their respective variations $(\nabla h_t, \nabla z_t)$ can be embedded in an augmented space:

$$\begin{pmatrix} h_t \\ \nabla h_t \\ z_t \\ \nabla z_t \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{I} & 0 & 0 & 0 & k_t v_t^\top \\ 0 & \mathbf{I} & 0 & 0 & \nabla k_t v_t^\top + k_t \nabla v_t^\top \\ 0 & 0 & \mathbf{I} & 0 & k_t \\ 0 & 0 & 0 & \mathbf{I} & \nabla k_t \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} h_{t-1} \\ \nabla h_{t-1} \\ z_{t-1} \\ \nabla z_{t-1} \\ 1 \end{pmatrix} \quad (32)$$

Since the transition blocks are identity matrices (or diagonal decays for RetNet/RWKV), TOSE applies without activation recomputation. The peak differentiation memory remains $O(D \cdot N)$, which is $O(1)$ relative to L .

E.2. Derivation for Decaying Recurrences (RWKV / RetNet)

RWKV-6 and RetNet introduce a time-varying or fixed decay α_t in the linear accumulation. The primal state follows $h_t = \alpha_t h_{t-1} + k_t v_t^\top$.

Total Differential: The variation ∇h_t must account for the sensitivity with respect to the decay parameter (if selective, as in RWKV-6):

$$\nabla h_t = \alpha_t \nabla h_{t-1} + (\nabla \alpha_t) h_{t-1} + \nabla k_t v_t^\top + k_t \nabla v_t^\top \quad (33)$$

where $\nabla\alpha_t$ is the variation of the decay factor. This matches the *Tangent Flow* structure in Equation (6). The term $(\nabla\alpha_t)h_{t-1}$ shows that PGF handles the selection mechanism in RWKV-6 with the same exactness as in Mamba.

E.3. Conclusion on Isomorphism

The mapping from Linear Transformers/RWKV to Phase Gradient Flow is structurally similar. Since Phase Gradient Flow is exact with $O(1)$ graph memory on Mamba (the superset), it provides a unified training framework for linear-time architectures. Models within the GLR family can scale to much longer contexts without hitting the reverse-mode AD (BPTT) memory wall.

F. Universality Verification on Toy Models

To verify the universality of Phase Gradient Flow beyond Mamba, we implement the block-wise differentiation protocol on three representative General Linear Recurrences: RWKV (linear attention with decay), RetNet (multi-head linear attention), and a discrete NeuralODE (Euler method).

Exactness Validation. We compare the parameter gradients accumulated via the block-wise numerical relay against full-sequence reverse-mode AD (BPTT) on sequences of length $L = 256$ with dimension $D = 64$. As shown below, the numerical relay yields bit-exact gradients (relative error $\sim 10^{-7}$, cosine similarity 1.0) across all models:

- **RWKV:** Rel. Error 2.61×10^{-7} , Cosine Sim 1.0000
- **RetNet:** Rel. Error 2.60×10^{-7} , Cosine Sim 1.0000
- **NeuralODE:** Rel. Error 1.63×10^{-7} , Cosine Sim 1.0000

Memory Scaling Analysis. We profile the peak computational graph memory as sequence length L scales from 256 to 8192. For reverse-mode AD, memory scales linearly with L . In contrast, our protocol maintains strict $O(1)$ graph memory scaling across all tested architectures:

- **RetNet (BPTT):** $L = 2048 \rightarrow 46.6$ MB
- **RetNet (Phase Gradient Flow):** $L = 8192 \rightarrow 29.3$ MB (Flat scaling)

This confirms that the memory wall reduction fundamentally applies to any model utilizing the GLR abstraction.

G. Second-Order Dynamical Isomorphism and Hessian Flow

The first-order exactness of PGF stems from Tangent-Flow Isomorphism. We extend the framework to second-order derivatives, where exact Hessian-vector products (HVPs) can be computed with $O(1)$ memory and $O(N)$ time.

G.1. Hessian Flow Derivation

Consider the primal GLR system $h_t = \bar{A}_t h_{t-1} + \mathbf{b}_t$. Let ∇_v and ∇_w be independent Fréchet variations along directions v and w .

Step 1: First-Order Recurrence (Shadow Flow). The variation $\nabla_v h_t$ evolves as:

$$\nabla_v h_t = \bar{A}_t (\nabla_v h_{t-1}) + \underbrace{(\nabla_v \bar{A}_t) h_{t-1} + \nabla_v \mathbf{b}_t}_{\text{Source}_v} \quad (34)$$

The recursive coefficient remains $\bar{A}_t$.

Step 2: Second-Order Recurrence (Hessian Flow). Applying ∇_w to the first-order flow using the Leibniz rule:

$$\nabla_{v,w}^2 h_t = \nabla_w [\bar{A}_t (\nabla_v h_{t-1}) + \text{Source}_v] \quad (35)$$

Expanding the terms:

$$\nabla_{v,w}^2 h_t = \bar{A}_t (\nabla_{v,w}^2 h_{t-1}) + (\nabla_w \bar{A}_t) (\nabla_v h_{t-1}) + \nabla_w (\text{Source}_v) \quad (36)$$

Define the *Second-Order Source Term* H_t :

$$\nabla^2 h_t = \bar{A}_t \nabla^2 h_{t-1} + H_t \quad (37)$$

where H_t decomposes into Cross-Talk and Curvature components:

$$H_t = \underbrace{(\nabla_w \bar{A}_t)(\nabla_v h_{t-1}) + (\nabla_v \bar{A}_t)(\nabla_w h_{t-1})}_{\text{Cross-Talk (First-order Interactions)}} + \underbrace{(\nabla_{v,w}^2 \bar{A}_t) h_{t-1} + \nabla_{v,w}^2 \mathbf{b}_t}_{\text{Curvature Driving Term}} \quad (38)$$

G.2. Theoretical Implications: Triple-Scan Newton Methods

Equation (37) shows that the second-order derivative $\nabla^2 h_t$ evolves as a higher-order flow with the same transition operator $\bar{A}_t$ as the primal system.

1. *O(1) Memory*: Computing $\nabla^2 h_t$ requires only local states $(h_{t-1}, \nabla_v h_{t-1}, \nabla_w h_{t-1})$ and local parameter curvatures. After each TOSE block, history can be discarded. 2. *Computational Complexity*: The exact Hessian calculation reduces to three parallel scans:

$$\text{Scan}(h) \rightarrow \text{Scan}(\nabla h) \rightarrow \text{Scan}(\nabla^2 h)$$

The total complexity is $3 \times O(N)$. Second-order optimization (e.g., Newton-Mamba) therefore scales linearly with sequence length.